**_Schmidtea mediterranea_ hypoxia time-course bulk ATAC-seq analysis workflow**

**Trimming of ATAC-seq reads**

```
for sample in 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2
24hR3 36hR1 36hR2 36hR3
do
  cutadapt \
    -a CTGTCTCTTATACACATCT \
    -A CTGTCTCTTATACACATCT \
    -q 20,20 \
    -m 20 \
    -o ${sample}_1_trimmed.fq \
    -p ${sample}_2_trimmed.fq \
    ${sample}_1.fastq.gz ${sample}_2.fastq.gz
done
```

**Alignment of ATAC-seq reads to Smed_rink.fa**

```
for sample in 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2
24hR3 36hR1 36hR2 36hR3 do bowtie2 --very-sensitive -X 2000 -p 8 -x Smed_Rink \ -1
${sample}_1_trimmed.fq \ -2 ${sample}_2_trimmed.fq \ | samtools view -bS - \ |
samtools sort -o ${sample}.sorted.bam - && samtools index ${sample}.sorted.bam
done
```

**Obtaining alignment summary of all ATAC-seq samples**

```
(ATAC-seq) lady7197@Bio-L-SV-
002001:/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment$ cat > atac_alignment_summary.py <<'PY'
> #!/usr/bin/env python3
>
> from pathlib import Path
> import csv
> import re
> import subprocess
> import sys
>
>
> def require_tool(tool: str) -> None:
>     from shutil import which
>     if which(tool) is None:
>         print(f"Error: '{tool}' is not on PATH.", file=sys.stderr)
>         sys.exit(1)
>
>
> def run_cmd(cmd: list[str]) -> str:
```

```
>     try:
>         result = subprocess.run(cmd, check=True, capture_output=True, text=True)
>         return result.stdout
>     except subprocess.CalledProcessError as e:
>         print(f"Error running: {' '.join(cmd)}", file=sys.stderr)
>         if e.stderr:
>             print(e.stderr, file=sys.stderr)
>         sys.exit(1)
>
>
> def find_int(pattern: str, text: str) -> int:
>     m = re.search(pattern, text, flags=re.MULTILINE)
>     return int(m.group(1)) if m else 0
>
>
> def find_pair_int(pattern: str, text: str) -> tuple[int, int]:
>     m = re.search(pattern, text, flags=re.MULTILINE)
>     if not m:
>         return 0, 0
>     return int(m.group(1)), int(m.group(2))
>
>
> def parse_flagstat(flagstat_text: str) -> dict[str, int]:
>     stats = {}
>     stats["total_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+in total\b", flagstat_text)
>     stats["secondary_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+secondary\b",
flagstat_text)
>     stats["supplementary_reads"] =
find_int(r"^\s*(\d+)\s+\+\s+\d+\s+supplementary\b", flagstat_text)
>     stats["duplicates_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+duplicates\b",
flagstat_text)
>     stats["mapped_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+mapped\b",
flagstat_text)
>     stats["paired_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+paired in sequencing\b",
flagstat_text)
>     stats["read1_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+read1\b", flagstat_text)
>     stats["read2_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+read2\b", flagstat_text)
>     stats["properly_paired_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+properly
paired\b", flagstat_text)
>     stats["with_itself_mate_mapped"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+with itself
and mate mapped\b", flagstat_text)
>     stats["singletons_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+singletons\b",
flagstat_text)
>     stats["mate_diff_chr_mapq5_reads"] = find_int(r"^\s*(\d+)\s+\+\s+\d+\s+with mate
mapped to a different chr \(mapQ>=5\)\b", flagstat_text)
>     return stats
>
```

```
>
> def calc_pct(num: int, denom: int) -> float:
>     return (100.0 * num / denom) if denom else 0.0
>
>
> def main() -> None:
>     require_tool("samtools")
>
>     bam_files = sorted(Path(".").glob("*.sorted.bam"))
>     if not bam_files:
>         print("No *.sorted.bam files found in the current directory.", file=sys.stderr)
>         sys.exit(1)
>
>     out_path = Path("atac_alignment_summary.tsv")
>
>     header = [
>         "sample",
>         "bam_file",
>         "total_reads",
>         "mapped_reads",
>         "mapped_pct",
>         "paired_reads",
>         "properly_paired_reads",
>         "properly_paired_pct",
>         "duplicates_reads",
>         "duplicates_pct",
>         "singletons_reads",
>         "singletons_pct",
>         "secondary_reads",
>         "supplementary_reads",
>         "read1_reads",
>         "read2_reads",
>         "with_itself_mate_mapped_reads",
>         "mate_diff_chr_mapq5_reads",
>         "chrM_mapped_reads",
>         "chrM_pct_of_mapped",
>     ]
>
>     with out_path.open("w", newline="") as out_f:
>         writer = csv.writer(out_f, delimiter="\t")
>         writer.writerow(header)
>         out_f.flush()
>
>         for i, bam in enumerate(bam_files, start=1):
>             sample = bam.name.replace(".sorted.bam", "")
>
>             flagstat = run_cmd(["samtools", "flagstat", str(bam)])
```

```
>         fs = parse_flagstat(flagstat)
>
>         idxstats = run_cmd(["samtools", "idxstats", str(bam)])
>         chrM_mapped = 0
>         for line in idxstats.strip().splitlines():
>           fields = line.split("\t")
>           if len(fields) >= 3 and fields[0] in {"MT", "chrM", "M",
"mitochondrion_genome"}:
>             try:
>               chrM_mapped += int(fields[2])
>             except ValueError:
>               pass
>
>         mapped_pct = calc_pct(fs["mapped_reads"], fs["total_reads"])
>         properly_paired_pct = calc_pct(fs["properly_paired_reads"], fs["total_reads"])
>         duplicates_pct = calc_pct(fs["duplicates_reads"], fs["total_reads"])
>         singletons_pct = calc_pct(fs["singletons_reads"], fs["total_reads"])
>         chrM_pct_of_mapped = calc_pct(chrM_mapped, fs["mapped_reads"])
>
>         writer.writerow([
>           sample,
>           bam.name,
>           fs["total_reads"],
>           fs["mapped_reads"],
>           f"{mapped_pct:.2f}",
>           fs["paired_reads"],
>           fs["properly_paired_reads"],
>           f"{properly_paired_pct:.2f}",
>           fs["duplicates_reads"],
>           f"{duplicates_pct:.2f}",
>           fs["singletons_reads"],
>           f"{singletons_pct:.2f}",
>           fs["secondary_reads"],
>           fs["supplementary_reads"],
>           fs["read1_reads"],
>           fs["read2_reads"],
>           fs["with_itself_mate_mapped"],
>           fs["mate_diff_chr_mapq5_reads"],
>           chrM_mapped,
>           f"{chrM_pct_of_mapped:.2f}",
>         ])
>         out_f.flush()
>
>         print(f"[{i}/{len(bam_files)}] finished {sample}", flush=True)
>
>   print(f"Done. Wrote {out_path.resolve()}")
>
```

```
>
> if __name__ == "__main__":
>     main()
> PY
(ATAC-seq) lady7197@Bio-L-SV-002001:/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/alignment$
(ATAC-seq) lady7197@Bio-L-SV-002001:/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/alignment$ python3 -u atac_alignment_summary.py
[1/15] finished 0hR1
[2/15] finished 0hR2
[3/15] finished 0hR3
[4/15] finished 12hR1
[5/15] finished 12hR2
[6/15] finished 12hR3
[7/15] finished 24hR1
[8/15] finished 24hR2
[9/15] finished 24hR3
[10/15] finished 36hR1
[11/15] finished 36hR2
[12/15] finished 36hR3
[13/15] finished 3hR1
[14/15] finished 3hR2
[15/15] finished 3hR3
Done. Wrote /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/alignment/atac_alignment_summary.tsv
```

**Conducting ATAC filtering**

```
for bam in *.sorted.bam; do
  sample=${bam%.sorted.bam}

  echo "Processing $sample"

  samtools view -b -f 2 -q 30 "$bam" > "${sample}.filtered.bam"
  samtools sort -o "${sample}.filtered.sorted.bam" "${sample}.filtered.bam"
  samtools index "${sample}.filtered.sorted.bam"

  rm "${sample}.filtered.bam"
done
```

**Filtering out mitochondrial reads**

Smed_Rink.fa already excludes mitochondrial contigs (Pérez-Posada *et al.*, 2025).

**Calculating unique mapping rate**

```
cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/alignment

echo -e "Sample\tProperlyPairedReads\tUniqueMappedReads_MAPQ30\tUniqueMappingRatePct" \
> unique_mapping_rate.tsv

for bam in *.sorted.bam
do

  sample=${bam%.sorted.bam}


##########################################################################
  # ALL PROPERLY PAIRED READS

##########################################################################

  proper=$(samtools view -@ 8 -c -f 2 "$bam")


##########################################################################
  # UNIQUE PROPERLY PAIRED READS
  # MAPQ >=30

##########################################################################

  unique=$(samtools view -@ 8 -c -f 2 -q 30 "$bam")


##########################################################################
  # UNIQUE MAPPING RATE

##########################################################################

  rate=$(awk -v u="$unique" -v p="$proper" \
  'BEGIN{printf "%.2f", (u/p)*100}')
```

```
##########################################################################
######
  # OUTPUT

##########################################################################
######

  echo -e "${sample}\t${proper}\t${unique}\t${rate}" \
  | tee -a unique_mapping_rate.tsv

done
```

**Conducting de-duplication**

```
mkdir -p dedup_bam_filtered tmp_fixmate

for bam in *.filtered.sorted.bam; do
  [ -e "$bam" ] || continue

  sample=${bam%.filtered.sorted.bam}
  echo "Processing $sample at $(date)"

  samtools sort -@ 16 -n -o tmp_fixmate/${sample}.name_sorted.bam "$bam"
  samtools fixmate -@ 16 -m tmp_fixmate/${sample}.name_sorted.bam
tmp_fixmate/${sample}.fixmate.bam
  samtools sort -@ 16 -o tmp_fixmate/${sample}.positionsorted.bam
tmp_fixmate/${sample}.fixmate.bam
  samtools markdup -@ 16 -r tmp_fixmate/${sample}.positionsorted.bam
dedup_bam_filtered/${sample}.dedup.bam
  samtools index dedup_bam_filtered/${sample}.dedup.bam

  rm -f tmp_fixmate/${sample}.name_sorted.bam \
      tmp_fixmate/${sample}.fixmate.bam \
      tmp_fixmate/${sample}.positionsorted.bam

  echo "Finished $sample at $(date)"
done

cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment

echo -e
"Sample\tPreDedupReads\tPostDedupReads\tPercentRetained\tPercentDuplicatesRe
moved"

for dedup in dedup_bam_filtered/*.dedup.bam
do
```

```
  sample=$(basename "$dedup" .dedup.bam)

  original="${sample}.filtered.sorted.bam"

  pre=$(samtools view -@ 8 -c "$original")

  post=$(samtools view -@ 8 -c "$dedup")

  retained=$(awk -v p="$post" -v q="$pre" 'BEGIN{printf "%.2f", (p/q)*100}')

  removed=$(awk -v p="$post" -v q="$pre" 'BEGIN{printf "%.2f", 100-((p/q)*100)}')

  echo -e "${sample}\t${pre}\t${post}\t${retained}\t${removed}"

done

cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment

echo -e
"Sample\tTotalReads\tFilteredReads\tDeduplicatedReads\tFilteringRetentionPct\tFina
lRetentionPct"

for dedup in dedup_bam_filtered/*.dedup.bam
do

  sample=$(basename "$dedup" .dedup.bam)

  totalbam="${sample}.sorted.bam"

  filteredbam="${sample}.filtered.sorted.bam"


#########################################################################
######
  # COUNTS

#########################################################################
######

  total=$(samtools view -@ 8 -c "$totalbam")

  filtered=$(samtools view -@ 8 -c "$filteredbam")

  deduped=$(samtools view -@ 8 -c "$dedup")
```

```
#################################################################################
  # PERCENTAGES

#################################################################################

  filter_pct=$(awk -v f="$filtered" -v t="$total" \
  'BEGIN{printf "%.2f", (f/t)*100}')

  final_pct=$(awk -v d="$deduped" -v t="$total" \
  'BEGIN{printf "%.2f", (d/t)*100}')


#################################################################################
  # OUTPUT

#################################################################################

  echo -e "${sample}\t${total}\t${filtered}\t${deduped}\t${filter_pct}\t${final_pct}"

done \
| tee ATAC_final_mapping_summary.tsv

cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment
```

**Calculating fragment size distribution**

```
cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment

mkdir -p fragment_size_distribution/plots

for bam in dedup_bam/*.dedup.bam; do
  sample=$(basename "$bam" .dedup.bam)

  samtools view -f 66 -F 3852 "$bam" \
   | awk '{
       t = $9
       if (t < 0) t = -t
       if (t > 0 && t < 1000) print t
     }' > fragment_size_distribution/${sample}.fragment_lengths.txt
done
```

```
cat > plot_fragments.py << 'EOF'
import os
import glob
import matplotlib.pyplot as plt

outdir = "fragment_size_distribution"
plotdir = os.path.join(outdir, "plots")
os.makedirs(plotdir, exist_ok=True)

files = sorted(glob.glob(os.path.join(outdir, "*.fragment_lengths.txt")))
all_data = {}

for f in files:
    sample = os.path.basename(f).replace(".fragment_lengths.txt", "")
    vals = []

    with open(f) as fh:
        for line in fh:
            try:
                x = int(line.strip())
                if 0 < x < 1000:
                    vals.append(x)
            except ValueError:
                pass

    all_data[sample] = vals

    plt.figure(figsize=(8, 5))
    plt.hist(vals, bins=200, range=(0, 800))
    plt.xlabel("Fragment length (bp)")
    plt.ylabel("Count")
    plt.title(sample)
    plt.tight_layout()
    plt.savefig(os.path.join(plotdir, f"{sample}.png"), dpi=200)
    plt.close()

plt.figure(figsize=(10, 7))

for sample, vals in all_data.items():
    plt.hist(
        vals,
        bins=200,
        range=(0, 800),
        histtype="step",
        linewidth=1.5,
        density=True,
```

```
    label=sample,
  )

plt.xlabel("Fragment length (bp)")
plt.ylabel("Density")
plt.title("Fragment size distribution across all samples")
plt.legend(fontsize=7, ncol=2)
plt.tight_layout()
plt.savefig(os.path.join(plotdir, "combined_fragment_size_distribution.png"), dpi=200)
plt.close()
EOF

python3 plot_fragments.py

ls -lh fragment_size_distribution
ls -lh fragment_size_distribution/plots
```

**Plotting combined fragment size distribution**

```
/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/alignment

cat > make_fragment_style_plot.py <<'EOF'
import os
import re
import glob
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

outdir = "fragment_size_distribution"
plotdir = os.path.join(outdir, "plots")
os.makedirs(plotdir, exist_ok=True)

files = sorted(glob.glob(os.path.join(outdir, "*.fragment_lengths.txt")))
if not files:
  raise SystemExit("No fragment length files found")

def sort_key(path):
  sample = os.path.basename(path).replace(".fragment_lengths.txt", "")
  m = re.match(r"^(\d+)hR(\d+)$", sample)
  if m:
    return (int(m.group(1)), int(m.group(2)), sample)
  return (9999, 9999, sample)

files = sorted(files, key=sort_key)
samples = [os.path.basename(f).replace(".fragment_lengths.txt", "") for f in files]
```

```
bin_edges = np.arange(0, 501, 1)
x = np.arange(0.5, 500.0, 1.0)

all_hist = []

for f in files:
   vals = np.loadtxt(f, dtype=float)
   if vals.ndim == 0:
      vals = np.array([vals], dtype=float)

   vals = vals[(vals >= 0) & (vals <= 1000)]

   hist, _ = np.histogram(vals, bins=bin_edges, density=True)

   # start curves at 35 bp
   hist = hist.astype(float)
   hist[x < 35] = np.nan

   all_hist.append(hist)

# red-blue palette
colors = cm.get_cmap("RdBu")(np.linspace(0.05, 0.95, len(samples)))

plt.figure(figsize=(12, 12))
ax = plt.gca()

for sample, hist, color in zip(samples, all_hist, colors):
   ax.plot(x, hist, color=color, linewidth=3.2, alpha=0.9, label=sample)

ax.set_xlim(0, 500)
ax.set_xlabel("\nFragment length (bp)", fontsize=20)
ax.set_ylabel("Normalised read density\n", fontsize=20)

ax.tick_params(axis="x", labelsize=15)
ax.tick_params(axis="y", labelsize=15)

for xpos in [50, 150, 350]:
   ax.axvline(xpos, linestyle="dashed", color="black", linewidth=1)

leg = ax.legend(
   title="Sample",
   bbox_to_anchor=(1.02, 0.5),
   loc="center left",
   frameon=False,
   ncol=1,
   fontsize=14,
   title_fontsize=16
```

```
)

for line in leg.get_lines():
   line.set_linewidth(3.2)

plt.tight_layout(rect=[0, 0, 0.78, 1])
plt.savefig(os.path.join(plotdir, "combined_fragment_size_distribution.png"), dpi=300)

print("Done")
EOF

python3 make_fragment_style_plot.py
```

**Plotting ATAC-seq sample-to-sample correlation heatmap**

```
conda activate ATAC-seq
cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment

mkdir -p tss_enrichment/correlation

multiBigwigSummary bins \
  -b tss_enrichment/bigwig/0hR1.bw \
    tss_enrichment/bigwig/0hR2.bw \
    tss_enrichment/bigwig/0hR3.bw \
    tss_enrichment/bigwig/3hR1.bw \
    tss_enrichment/bigwig/3hR2.bw \
    tss_enrichment/bigwig/3hR3.bw \
    tss_enrichment/bigwig/12hR1.bw \
    tss_enrichment/bigwig/12hR2.bw \
    tss_enrichment/bigwig/12hR3.bw \
    tss_enrichment/bigwig/24hR1.bw \
    tss_enrichment/bigwig/24hR2.bw \
    tss_enrichment/bigwig/24hR3.bw \
    tss_enrichment/bigwig/36hR1.bw \
    tss_enrichment/bigwig/36hR2.bw \
    tss_enrichment/bigwig/36hR3.bw \
  --labels 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2 24hR3 36hR1
36hR2 36hR3 \
  -o tss_enrichment/correlation/ATAC_bigwig_summary.npz

plotCorrelation \
  -in tss_enrichment/correlation/ATAC_bigwig_summary.npz \
  --corMethod spearman \
  --whatToPlot heatmap \
  --plotNumbers \
  --skipZeros \
```

```
  --labels 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2 24hR3 36hR1
36hR2 36hR3 \
  --outFileCorMatrix tss_enrichment/correlation/ATAC_correlation_matrix.tsv \
  -o tss_enrichment/correlation/ATAC_correlation_heatmap.pdf

ls -lh tss_enrichment/correlation/ATAC_correlation_heatmap.pdf
```

**Plotting ATAC-seq PCA plot**

```
cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis/alignment
mkdir -p tss_enrichment/pca

multiBigwigSummary bins \
  -b tss_enrichment/bigwig/0hR1.bw \
    tss_enrichment/bigwig/0hR2.bw \
    tss_enrichment/bigwig/0hR3.bw \
    tss_enrichment/bigwig/3hR1.bw \
    tss_enrichment/bigwig/3hR2.bw \
    tss_enrichment/bigwig/3hR3.bw \
    tss_enrichment/bigwig/12hR1.bw \
    tss_enrichment/bigwig/12hR2.bw \
    tss_enrichment/bigwig/12hR3.bw \
    tss_enrichment/bigwig/24hR1.bw \
    tss_enrichment/bigwig/24hR2.bw \
    tss_enrichment/bigwig/24hR3.bw \
    tss_enrichment/bigwig/36hR1.bw \
    tss_enrichment/bigwig/36hR2.bw \
    tss_enrichment/bigwig/36hR3.bw \
  --labels 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2 24hR3 36hR1
36hR2 36hR3 \
  -o tss_enrichment/pca/ATAC_summary.npz

plotPCA \
  -in tss_enrichment/pca/ATAC_summary.npz \
  -o tss_enrichment/pca/ATAC_PCA.pdf \
  --labels 0hR1 0hR2 0hR3 3hR1 3hR2 3hR3 12hR1 12hR2 12hR3 24hR1 24hR2 24hR3 36hR1
36hR2 36hR3

ls -lh tss_enrichment/pca/ATAC_PCA.pdf

python - <<'PY'
import re
import numpy as np
import matplotlib.pyplot as plt

npz_file = "pca/ATAC_summary.npz"
```

```
out_pdf = "pca/ATAC_PCA_timepoint_colored.pdf"
out_png = "pca/ATAC_PCA_timepoint_colored.png"

npz = np.load(npz_file, allow_pickle=True)

# Load matrix
X = None
for k in ("matrix", "readCounts", "counts"):
   if k in npz.files:
      X = np.asarray(npz[k], dtype=float)
      break
if X is None:
   raise KeyError(f"No matrix-like array found in {npz.files}")

# Load labels
labels = None
for k in ("labels", "sampleLabels"):
   if k in npz.files:
      labels = [str(x) for x in npz[k]]
      break
if labels is None:
   raise KeyError(f"No labels found in {npz.files}")

# Ensure samples are rows
if X.shape[0] != len(labels) and X.shape[1] == len(labels):
   X = X.T
if X.shape[0] != len(labels):
   raise ValueError(f"Shape mismatch: X={X.shape}, labels={len(labels)}")

# Clean and remove constant bins
X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)
keep = X.var(axis=0) > 0
X = X[:, keep]

# Z-score each bin across samples so PCA reflects sample structure
mu = X.mean(axis=0)
sd = X.std(axis=0, ddof=1)
sd[sd == 0] = 1.0
Xz = (X - mu) / sd

# PCA via SVD
U, S, Vt = np.linalg.svd(Xz, full_matrices=False)
pcs = U[:, :2] * S[:2]

eigvals = (S**2) / (Xz.shape[0] - 1)
var_ratio = eigvals / eigvals.sum()
```

```
def get_timepoint(label):
   m = re.match(r"(\d+h)", label)
   return m.group(1) if m else label

timepoints = [get_timepoint(x) for x in labels]

# palette colors = {
   "0h":  "#000000",  # black
   "3h":  "#009E73",  # green
   "12h": "#56B4E9",  # light blue
   "24h": "#D2B48C",  # light brown/tan
   "36h": "#7B3294",  # purple
}
order = ["0h", "3h", "12h", "24h", "36h"]

fig, ax = plt.subplots(figsize=(8.2, 6.2))

for tp in order:
   idx = [i for i, t in enumerate(timepoints) if t == tp]
   if not idx:
      continue
   ax.scatter(
      pcs[idx, 0],
      pcs[idx, 1],
      s=55,
      marker="o",
      c=colors[tp],
      edgecolors="none",
      label=tp,
      alpha=0.95,
   )

ax.axhline(0, color="black", linestyle=":", linewidth=1)
ax.axvline(0, color="black", linestyle=":", linewidth=1)

ax.set_xlabel(f"PC1 ({var_ratio[0]*100:.1f}% of variance explained)")
ax.set_ylabel(f"PC2 ({var_ratio[1]*100:.1f}% of variance explained)")
ax.set_title("ATAC-seq PCA")

ax.legend(title="Timepoint", frameon=True, loc="center left", bbox_to_anchor=(1.02, 0.5))
plt.tight_layout()

plt.savefig(out_pdf)
plt.savefig(out_png, dpi=300)

print("Overwritten:")
print(out_pdf)
```

```
print(out_png)
PY
```

**Calculating fraction of reads in peaks (FRiP) score**

```
conda create -n frip_env -c bioconda -c conda-forge subread -y
conda activate frip_env

which featureCounts

BASE=/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis

rm -rf \
  "${BASE}/FrIP/frip_saf" \
  "${BASE}/FrIP/frip_counts" \
  "${BASE}/FrIP/frip_scores.txt"

cat > "${BASE}/FrIP/run_frip.sh" <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

# FRiP calculation script
#
# FRiP = fraction of reads in peaks
# Calculated as:
#   Assigned reads / total reads
# where totals are derived from featureCounts summary output

BASE=/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis
INPUT=${BASE}/alignment/dedup_bam_filtered
PEAKS=${BASE}/alignment/tss_enrichment/peaks
OUTDIR=${BASE}/FrIP

mkdir -p "${OUTDIR}/frip_saf" "${OUTDIR}/frip_counts"

# Check featureCounts exists
command -v featureCounts >/dev/null 2>&1 || {
  echo "ERROR: featureCounts not found. Activate frip_env." >&2
  exit 1
}

tmp_scores="${OUTDIR}/frip_scores.tmp.tsv"
echo -e "Sample\tFRiP" > "$tmp_scores"

shopt -s nullglob
```

```
for peak in "${PEAKS}"/*_peaks.narrowPeak; do

 sample=$(basename "$peak" _peaks.narrowPeak)
 bam="${INPUT}/${sample}.dedup.bam"
 saf="${OUTDIR}/frip_saf/${sample}.saf"
 out="${OUTDIR}/frip_counts/${sample}.txt"

 # Sanity check
 if [[ ! -f "$bam" ]]; then
  echo "ERROR: Missing BAM file: $bam" >&2
  exit 1
 fi

 # Create SAF annotation from peaks
 awk 'BEGIN{OFS="\t"; print "GeneID","Chr","Start","End","Strand"}
     {print "peak_"NR,$1,$2+1,$3,"."}' "$peak" > "$saf"

 npeaks=$(wc -l < "$saf")
 echo "Processing ${sample} (${npeaks} peaks)"

 # Count reads in peaks
 featureCounts \
  -F SAF \
  -a "$saf" \
  -o "$out" \
  -p \
  --countReadPairs \
  "$bam" >/dev/null

 # Calculate FRiP
 assigned=$(awk '$1=="Assigned"{print $2}' "${out}.summary")
 total=$(awk 'NR>1 {sum+=$2} END{print sum+0}' "${out}.summary")

 if [[ -z "${assigned}" || "${total}" -eq 0 ]]; then
  echo "ERROR: Could not parse featureCounts summary for ${sample}" >&2
  exit 1
 fi

 frip=$(awk -v a="$assigned" -v t="$total" 'BEGIN{printf "%.4f", a/t}')

 echo -e "${sample}\t${frip}" >> "$tmp_scores"

done

# Sort output (natural order)
{
```

```
  head -n 1 "$tmp_scores"
  tail -n +2 "$tmp_scores" | sort -V
} > "${OUTDIR}/frip_scores.txt"

rm -f "$tmp_scores"

echo "FRiP calculation complete:"
echo "${OUTDIR}/frip_scores.txt"
EOF

chmod +x "${BASE}/FrIP/run_frip.sh"

cd "${BASE}/alignment"
conda activate frip_env

"${BASE}/FrIP/run_frip.sh"

cat "${BASE}/FrIP/frip_scores.txt"
```

**Conducting DiffBind analysis**

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(DiffBind)
  library(dplyr)
})

# ============================================================
# DiffBind consensus peakset generation
#
# Input:
#   - diffbind_samplesheet.csv
#   - alignment/dedup_bam_filtered/*.dedup.bam
#   - alignment/tss_enrichment/peaks/*_peaks.narrowPeak
#
# Output:
#   - diffbind_samplesheet_absolute.csv
#   - consensus_peakset.csv
#   - consensus_peakset.bed
#
# ============================================================

# ----------------------------
# Paths
# ----------------------------
```

```
base_dir <- "/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-
course/april_2026_analysis"

align_dir <- file.path(base_dir, "alignment")

input_sheet <- file.path(
  base_dir,
  "DiffBind",
  "diffbind_samplesheet.csv"
)

outdir <- file.path(
  base_dir,
  "DiffBind",
  "consensus_peakset"
)

dir.create(outdir, showWarnings = FALSE, recursive = TRUE)

abs_sheet <- file.path(
  outdir,
  "diffbind_samplesheet_absolute.csv"
)

# ----------------------------
# Building absolute sample sheet
# ----------------------------
message("Building absolute sample sheet...")

sheet <- read.csv(
  input_sheet,
  stringsAsFactors = FALSE
)

required_cols <- c(
  "SampleID",
  "Tissue",
  "Condition",
  "Replicate",
  "bamReads",
  "Peaks",
  "PeakCaller"
)

missing_cols <- setdiff(
  required_cols,
  names(sheet)
```

```
)

if(length(missing_cols) > 0) {
  stop(
    "Input sample sheet is missing columns: ",
    paste(missing_cols, collapse = ", ")
  )
}

# Absolute BAM paths
sheet$bamReads <- file.path(
  align_dir,
  "dedup_bam_filtered",
  paste0(sheet$SampleID, ".dedup.bam")
)

# Absolute peak paths
sheet$Peaks <- file.path(
  align_dir,
  "tss_enrichment/peaks",
  paste0(sheet$SampleID, "_peaks.narrowPeak")
)

write.csv(
  sheet,
  abs_sheet,
  row.names = FALSE
)

# ------------------------------
# Building DiffBind object
# ------------------------------
message("Building DiffBind object...")

db <- dba(
  sampleSheet = abs_sheet
)

# ------------------------------
# Creating consensus peakset
# ------------------------------
message("Generating consensus peakset...")

db <- dba.count(
  db,
  minOverlap = 2,
  summits = 250
```

```
)

# ----------------------------
# Extracting consensus peaks
# ----------------------------
message("Exporting consensus peakset...")

consensus_peaks <- dba.peakset(
  db,
  bRetrieve = TRUE
)

consensus_df <- as.data.frame(
  consensus_peaks
)

# CSV export
write.csv(
  consensus_df,
  file.path(
    outdir,
    "consensus_peakset.csv"
  ),
  row.names = FALSE
)

# BED export
bed_df <- consensus_df[, c("Chr", "Start", "End")]

write.table(
  bed_df,
  file.path(
    outdir,
    "consensus_peakset.bed"
  ),
  sep = "\t",
  quote = FALSE,
  row.names = FALSE,
  col.names = FALSE
)

message("Consensus peakset generation complete.")
```

**Conducting TOBIAS analysis**

(Tobias_env) lady7197@Bio-L-SV-002001:/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS$

nano run_tobias_whitelist.sh

```
#!/usr/bin/env bash
set -euo pipefail

BASE=/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis
TOBIAS_DIR="$BASE/TOBIAS"
OUT="$BASE/TOBIAS/TOBIAS_hypoxia"
BAMDIR="$BASE/alignment/dedup_bam_filtered"
PEAKS="$BASE/hypoxia_ATAC_peaks.corrected.fixed.bed"
# (produced before by PEAKS=/drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/hypoxia_ATAC_peaks.corrected.bed
awk 'BEGIN{FS=OFS="\t"} NF>=3 {if ($2 < 0) $2 = 0; if ($3 > $2) print}' "$PEAKS" > /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/hypoxia_ATAC_peaks.corrected.fixed.bed) #

GENOME="$BASE/Smed_Rink.fa"
JASPAR_ALL="$BASE/TOBIAS/JASPAR2026_CORE_non-redundant_pfms_jaspar.txt"
FILTERED_MOTIFS="$OUT/hypoxia_ATAC_peaks.animal_only_whitelist.jaspar"

rm -rf "$OUT"/{merged_bam,ATACorrect,ScoreBigwig,BINDetect,plots,logs} "$FILTERED_MOTIFS"
mkdir -p "$OUT"/{merged_bam,ATACorrect,ScoreBigwig,BINDetect,plots,logs}

source "$(conda info --base)/etc/profile.d/conda.sh"
################################################################################
# Building a whitelist-filtered JASPAR file for TOBIAS BINDetect
################################################################################

python3 - "$JASPAR_ALL" "$METAZOAN_NON-REDUNDANT_MOTIFS" <<'PY'
import sys

jaspar_file = sys.argv[1]
filtered_out = sys.argv[2]

allowed_motif_ids = set([
```

"MA0010.1","MA0011.1","MA0012.1","MA0013.1","MA0015.1","MA0016.1","MA0022.1",
"MA0023.1","MA0026.1","MA0049.1",

"MA0085.1","MA0086.1","MA0094.1","MA0126.1","MA0165.1","MA0166.1","MA0167.1",
"MA0168.1","MA0169.1","MA0170.1",

"MA0171.1","MA0172.1","MA0173.1","MA0174.1","MA0175.1","MA0176.1","MA0177.1",
"MA0178.1","MA0179.1","MA0180.1",

"MA0181.1","MA0182.1","MA0183.1","MA0184.1","MA0185.1","MA0186.1","MA0187.1",
"MA0188.1","MA0189.1","MA0190.1",

"MA0191.1","MA0192.1","MA0193.1","MA0194.1","MA0195.1","MA0196.1","MA0197.1",
"MA0198.1","MA0199.1","MA0200.1",

"MA0201.1","MA0202.1","MA0203.1","MA0204.1","MA0205.1","MA0206.1","MA0207.1",
"MA0208.1","MA0209.1","MA0210.1",

"MA0211.1","MA0212.1","MA0213.1","MA0214.1","MA0215.1","MA0216.1","MA0217.1",
"MA0218.1","MA0219.1","MA0220.1",

"MA0221.1","MA0222.1","MA0224.1","MA0225.1","MA0226.1","MA0227.1","MA0228.1",
"MA0229.1","MA0230.1","MA0231.1",

"MA0232.1","MA0233.1","MA0234.1","MA0235.1","MA0236.1","MA0237.1","MA0238.1",
"MA0239.1","MA0240.1","MA0241.1",

"MA0242.1","MA0243.1","MA0244.1","MA0245.1","MA0246.1","MA0247.1","MA0248.1",
"MA0249.1","MA0250.1","MA0251.1",

"MA0252.1","MA0253.1","MA0254.1","MA0255.1","MA0256.1","MA0257.1","MA0094.2",
"MA0443.1","MA0444.1","MA0445.1",

"MA0446.1","MA0447.1","MA0448.1","MA0449.1","MA0450.1","MA0451.1","MA0452.1",
"MA0197.2","MA0454.1","MA0456.1",

"MA0457.1","MA0458.1","MA0459.1","MA0460.1","MA0529.1","MA0216.2","MA0530.1",
"MA0531.1","MA0452.2","MA0532.1",

"MA0533.1","MA0534.1","MA0237.2","MA0535.1","MA0536.1","MA0247.2","MA0915.1",
"MA0916.1","MA0917.1","MA0086.2",

"MA1455.1","MA1456.1","MA1457.1","MA1458.1","MA1459.1","MA1460.1","MA1461.1",
"MA1462.1","MA0529.2","MA0205.2",

"MA0249.2","MA1700.1","MA1702.1","MA1836.1","MA1837.1","MA1838.1","MA1839.1",
"MA1840.1","MA1841.1","MA2107.1",

"MA2108.1","MA2188.1","MA2189.1","MA2190.1","MA2191.1","MA2192.1","MA2193.1",
"MA2194.1","MA2195.1","MA2196.1",

"MA2197.1","MA2198.1","MA2199.1","MA2200.1","MA2201.1","MA2202.1","MA2203.1",
"MA2204.1","MA2205.1","MA2206.1",

"MA2207.1","MA2208.1","MA2209.1","MA2210.1","MA2211.1","MA2212.1","MA2213.1",
"MA2214.1","MA2215.1","MA2216.1",

"MA2217.1","MA2218.1","MA2219.1","MA2220.1","MA2221.1","MA2222.1","MA2223.1",
"MA2224.1","MA2225.1","MA2226.1",

"MA2227.1","MA2228.1","MA2229.1","MA2230.1","MA2231.1","MA2232.1","MA2233.1",
"MA2234.1","MA2235.1","MA2236.1",

"MA2237.1","MA2238.1","MA2239.1","MA2240.1","MA2241.1","MA2242.1","MA2243.1",
"MA2244.1","MA2245.1","MA2246.1",

"MA2247.1","MA2248.1","MA2249.1","MA2250.1","MA2251.1","MA2252.1","MA2253.1",
"MA2254.1","MA2255.1","MA2256.1",

"MA2257.1","MA2258.1","MA2259.1","MA2260.1","MA2261.1","MA2262.1","MA2263.1",
"MA2264.1","MA2265.1","MA2266.1",

"MA2267.1","MA2268.1","MA2269.1","MA2270.1","MA2271.1","MA2272.1","MA2273.1",
"MA2274.1","MA2275.1","MA2276.1",

"MA2277.1","MA2278.1","MA2279.1","MA2280.1","MA2281.1","MA2282.1","MA2283.1",
"MA2284.1","MA2285.1","MA2286.1",

"MA2287.1","MA2288.1","MA2289.1","MA2290.1","MA2291.1","MA2292.1","MA2293.1",
"MA2294.1","MA2463.1","MA2464.1",

"MA2465.1","MA2466.1","MA2467.1","MA2468.1","MA2295.1","MA2296.1","MA2297.1",
"MA2298.1","MA2299.1","MA2300.1",

"MA2301.1","MA2302.1","MA2303.1","MA2304.1","MA2305.1","MA2306.1","MA2307.1",
"MA2308.1","MA2309.1","MA2310.1",

"MA2311.1","MA2312.1","MA2313.1","MA2314.1","MA2315.1","MA2316.1","MA2317.1",
"MA2318.1","MA2319.1","MA2320.1",

"MA2321.1","MA2322.1","MA0529.3","MA0010.2","MA0011.2","MA0012.2","MA0443.2",
"MA0015.2","MA0184.2","MA0530.2",

"MA0531.2","MA0185.2","MA1455.2","MA1456.2","MA1836.2","MA0915.2","MA0189.2",
"MA1837.2","MA1838.2","MA0222.2",

"MA1839.2","MA1840.2","MA1457.2","MA1458.2","MA1841.2","MA0451.2","MA0452.3",
"MA0195.2","MA1459.2","MA0197.3",

"MA0126.2","MA1702.2","MA0457.2","MA1460.2","MA0536.2","MA0200.2","MA0239.2",
"MA0243.2","MA0458.2","MA0086.3",

"MA0532.2","MA1461.2","MA0247.3","MA0205.3","MA0249.3","MA0094.3","MA0016.2",
"MA1462.2","MA0252.2","MA0255.2",

"MA0257.2","MA0085.2","MA2630.1","MA2631.1","MA2632.1","MA2633.1","MA2316.2",
"MA0260.1","MA0261.1","MA0262.1",

"MA0263.1","MA0264.1","MA0537.1","MA0538.1","MA0541.1","MA0542.1","MA0543.1",
"MA0544.1","MA0545.1","MA0546.1",

"MA0547.1","MA0918.1","MA0919.1","MA0920.1","MA0921.1","MA0922.1","MA0923.1",
"MA0924.1","MA0925.1","MA0926.1",

"MA0927.1","MA0928.1","MA1438.1","MA1439.1","MA1440.1","MA1441.1","MA1442.1",
"MA1443.1","MA1444.1","MA1445.1",

"MA1446.1","MA1448.1","MA1449.1","MA1450.1","MA1451.1","MA1699.1","MA1701.1",
"MA1703.1","MA1704.1","MA2127.1",

"MA2128.1","MA2129.1","MA2130.1","MA2131.1","MA2132.1","MA2133.1","MA2134.1",
"MA2135.1","MA2136.1","MA2137.1",

"MA2138.1","MA2139.1","MA2140.1","MA2141.1","MA2142.1","MA2143.1","MA2144.1",
"MA2145.1","MA2146.1","MA2147.1",

"MA2148.1","MA2149.1","MA2150.1","MA2151.1","MA2152.1","MA2153.1","MA2154.1",
"MA2155.1","MA2156.1","MA2157.1",

"MA2158.1","MA2159.1","MA2160.1","MA2161.1","MA2162.1","MA2163.1","MA2164.1",
"MA2165.1","MA2166.1","MA2167.1",

"MA2168.1","MA2169.1","MA2170.1","MA2171.1","MA2172.1","MA2173.1","MA2174.1",
"MA2175.1","MA2176.1","MA2177.1",

"MA2178.1","MA2179.1","MA2180.1","MA2181.1","MA2182.1","MA2183.1","MA2184.1",
"MA2185.1","MA2186.1","MA2187.1",

"MA1438.2","MA1444.2","MA0264.2","MA1699.2","MA0921.2","MA0538.2","MA1446.2",
"MA0919.2","MA0541.2","MA1701.2",

"MA1439.2","MA0543.2","MA0920.2","MA0545.2","MA1449.2","MA0923.2","MA1441.2",
"MA1450.2","MA1703.2","MA0547.2",

"MA0925.2","MA1443.2","MA0918.2","MA0928.2","MA1704.2","MA0921.3","MA0118.1",
"MA1843.1","MA1844.1","MA1845.1",

"MA1846.1","MA1847.1","MA1848.1","MA1849.1","MA1850.1","MA1851.1","MA1852.1",
"MA1853.1","MA1854.1","MA1855.1",

"MA1856.1","MA1857.1","MA1858.1","MA1859.1","MA1860.1","MA1861.1","MA1862.1",
"MA1863.1","MA1864.1","MA1865.1",

"MA1866.1","MA1867.1","MA1868.1","MA1869.1","MA1870.1","MA1871.1","MA1872.1",
"MA1873.1","MA1874.1","MA1875.1",

"MA1876.1","MA1877.1","MA1878.1","MA1879.1","MA1880.1","MA1881.1","MA1882.1",
"MA1883.1","MA1884.1","MA1885.1",

"MA1886.1","MA1887.1","MA1888.1","MA1889.1","MA1890.1","MA1891.1","MA1892.1",
"MA1893.1","MA1894.1","MA1895.1",

"MA1896.1","MA1897.1","MA1898.1","MA1899.1","MA1900.1","MA1901.1","MA1902.1",
"MA1903.1","MA1904.1","MA1905.1",

"MA1906.1","MA1907.1","MA1908.1","MA1909.1","MA1910.1","MA1911.1","MA1912.1",
"MA1913.1","MA1914.1","MA1915.1",

"MA1916.1","MA1917.1","MA1918.1","MA1919.1","MA1920.1","MA1921.1","MA1922.1",
"MA1923.1","MA1924.1","MA1925.1",

"MA1926.1","MA1927.1","MA2109.1","MA2110.1","MA2111.1","MA2112.1","MA2113.1",
"MA2114.1","MA2115.1","MA2116.1",

"MA1843.2","MA1844.2","MA1845.2","MA1846.2","MA1847.2","MA1848.2","MA1849.2",
"MA1850.2","MA1851.2","MA1852.2",

"MA1853.2","MA1854.2","MA1855.2","MA1856.2","MA1857.2","MA1858.2","MA1859.2",
"MA1860.2","MA1861.2","MA1862.2",

"MA1863.2","MA1864.2","MA1865.2","MA1866.2","MA1867.2","MA1868.2","MA1869.2",
"MA1870.2","MA1871.2","MA1872.2",

"MA1873.2","MA1874.2","MA1875.2","MA1877.2","MA1878.2","MA1879.2","MA1880.2",
"MA1881.2","MA1882.2","MA1883.2",

"MA1884.2","MA1885.2","MA1886.2","MA1887.2","MA1888.2","MA1889.2","MA1891.2",
"MA1892.2","MA1894.2","MA1895.2",

"MA1896.2","MA0118.2","MA1897.2","MA1898.2","MA1899.2","MA1900.2","MA1901.2",
"MA1902.2","MA1903.2","MA1904.2",

"MA1905.2","MA1906.2","MA1907.2","MA1908.2","MA1909.2","MA1910.2","MA1911.2",
"MA1912.2","MA1913.2","MA1914.2",

"MA1915.2","MA1916.2","MA1917.2","MA1918.2","MA1919.2","MA1920.2","MA1921.2",
"MA1922.2","MA1923.2","MA1924.2",

"MA1925.2","MA1926.2","MA1927.2","MA0002.1","MA0003.1","MA0004.1","MA0006.1",
"MA0007.1","MA0009.1","MA0014.1",

"MA0017.1","MA0018.1","MA0019.1","MA0024.1","MA0025.1","MA0027.1","MA0028.1",
"MA0029.1","MA0030.1","MA0031.1",

"MA0032.1","MA0033.1","MA0035.1","MA0036.1","MA0037.1","MA0038.1","MA0039.1",
"MA0040.1","MA0041.1","MA0042.1",

"MA0043.1","MA0046.1","MA0047.1","MA0048.1","MA0050.1","MA0051.1","MA0052.1",
"MA0056.1","MA0058.1","MA0059.1",

"MA0060.1","MA0105.1","MA0062.1","MA0063.1","MA0065.1","MA0066.1","MA0067.1",
"MA0068.1","MA0069.1","MA0070.1",

"MA0071.1","MA0072.1","MA0073.1","MA0074.1","MA0075.1","MA0076.1","MA0077.1",
"MA0078.1","MA0079.1","MA0080.1",

"MA0081.1","MA0083.1","MA0084.1","MA0087.1","MA0088.1","MA0089.1","MA0090.1",
"MA0091.1","MA0092.1","MA0093.1",

"MA0095.1","MA0098.1","MA0099.1","MA0100.1","MA0101.1","MA0102.1","MA0103.1",
"MA0104.1","MA0105.2","MA0106.1",

"MA0107.1","MA0108.1","MA0108.2","MA0111.1","MA0112.1","MA0113.1","MA0114.1",
"MA0115.1","MA0116.1","MA0117.1",

"MA0119.1","MA0122.1","MA0124.1","MA0125.1","MA0130.1","MA0131.1","MA0132.1",
"MA0135.1","MA0136.1","MA0137.1",

"MA0138.1","MA0139.1","MA0140.1","MA0141.1","MA0142.1","MA0143.1","MA0144.1",
"MA0145.1","MA0146.1","MA0147.1",

"MA0148.1","MA0149.1","MA0062.2","MA0035.2","MA0039.2","MA0138.2","MA0002.2",
"MA0137.2","MA0104.2","MA0047.2",

"MA0112.2","MA0065.2","MA0150.1","MA0151.1","MA0152.1","MA0153.1","MA0154.1",
"MA0155.1","MA0156.1","MA0157.1",

"MA0158.1","MA0159.1","MA0160.1","MA0161.1","MA0162.1","MA0163.1","MA0164.1",
"MA0080.2","MA0018.2","MA0099.2",

"MA0079.2","MA0102.2","MA0258.1","MA0259.1","MA0442.1","MA0141.2","MA0143.2",
"MA0145.2","MA0146.2","MA0148.2",

"MA0461.1","MA0462.1","MA0463.1","MA0464.1","MA0465.1","MA0466.1","MA0467.1",
"MA0468.1","MA0469.1","MA0470.1",

"MA0471.1","MA0472.1","MA0473.1","MA0474.1","MA0475.1","MA0476.1","MA0477.1",
"MA0478.1","MA0479.1","MA0480.1",

"MA0481.1","MA0482.1","MA0483.1","MA0484.1","MA0485.1","MA0486.1","MA0488.1",
"MA0489.1","MA0490.1","MA0491.1",

"MA0492.1","MA0493.1","MA0494.1","MA0495.1","MA0496.1","MA0497.1","MA0498.1",
"MA0499.1","MA0500.1","MA0501.1",

"MA0502.1","MA0504.1","MA0505.1","MA0506.1","MA0507.1","MA0508.1","MA0509.1",
"MA0510.1","MA0511.1","MA0512.1",

"MA0514.1","MA0515.1","MA0516.1","MA0517.1","MA0518.1","MA0519.1","MA0520.1",
"MA0521.1","MA0522.1","MA0523.1",

"MA0524.1","MA0525.1","MA0526.1","MA0527.1","MA0528.1","MA0007.2","MA0102.3",
"MA0024.2","MA0154.2","MA0162.2",

"MA0076.2","MA0258.2","MA0098.2","MA0148.3","MA0035.3","MA0036.2","MA0037.2",
"MA0114.2","MA0050.2","MA0058.2",

"MA0052.2","MA0100.2","MA0147.2","MA0104.3","MA0150.2","MA0105.3","MA0060.2",
"MA0014.2","MA0080.3","MA0143.3",

"MA0079.3","MA0083.2","MA0137.3","MA0144.2","MA0140.2","MA0003.2","MA0106.2",
"MA0093.2","MA0095.2","MA0103.2",

"MA0591.1","MA0592.1","MA0593.1","MA0594.1","MA0595.1","MA0596.1","MA0597.1",
"MA0598.1","MA0599.1","MA0600.1",

"MA0113.2","MA0601.1","MA0602.1","MA0603.1","MA0604.1","MA0605.1","MA0606.1",
"MA0607.1","MA0608.1","MA0609.1",

"MA0610.1","MA0611.1","MA0612.1","MA0613.1","MA0614.1","MA0615.1","MA1099.1",
"MA0616.1","MA0618.1","MA0619.1",

"MA0620.1","MA0621.1","MA0622.1","MA0623.1","MA0624.1","MA0625.1","MA0626.1",
"MA0627.1","MA0628.1","MA0629.1",

"MA0630.1","MA0631.1","MA0632.1","MA0633.1","MA0634.1","MA0007.3","MA0635.1",
"MA0636.1","MA0638.1","MA0639.1",

"MA0154.3","MA0598.2","MA0640.1","MA0641.1","MA0136.2","MA0027.2","MA0642.1",
"MA0643.1","MA0644.1","MA0645.1",

"MA0475.2","MA0042.2","MA0033.2","MA0157.2","MA0646.1","MA0647.1","MA0648.1",
"MA0649.1","MA0131.2","MA0043.2",

"MA0046.2","MA0153.2","MA0650.1","MA0651.1","MA0486.2","MA0652.1","MA0653.1",
"MA0654.1","MA0655.1","MA0656.1",

"MA0657.1","MA0658.1","MA0659.1","MA0660.1","MA0661.1","MA0662.1","MA0663.1",
"MA0664.1","MA0665.1","MA0666.1",

"MA0667.1","MA0668.1","MA0669.1","MA0670.1","MA0671.1","MA0048.2","MA0672.1",
"MA0673.1","MA0674.1","MA0675.1",

"MA0676.1","MA0677.1","MA0678.1","MA0679.1","MA0068.2","MA0680.1","MA0681.1",
"MA0682.1","MA0683.1","MA0075.2",

"MA0684.1","MA0512.2","MA0685.1","MA0686.1","MA0080.4","MA0687.1","MA0083.3",
"MA0009.2","MA0688.1","MA0689.1",

"MA0690.1","MA0090.2","MA0691.1","MA0692.1","MA0693.1","MA0694.1","MA0695.1",
"MA0696.1","MA0697.1","MA0698.1",

"MA0699.1","MA0700.1","MA0701.1","MA0702.1","MA0703.1","MA0704.1","MA0705.1",
"MA0706.1","MA0707.1","MA0708.1",

"MA0709.1","MA0122.2","MA0710.1","MA0124.2","MA0711.1","MA0712.1","MA0132.2",
"MA0713.1","MA0714.1","MA0715.1",

"MA0716.1","MA0717.1","MA0718.1","MA0719.1","MA0720.1","MA0721.1","MA0722.1",
"MA0723.1","MA0724.1","MA0725.1",

"MA0726.1","MA0112.3","MA0141.3","MA0592.2","MA0114.3","MA0017.2","MA0113.3",
"MA0727.1","MA0728.1","MA0729.1",

"MA0730.1","MA0731.1","MA0472.2","MA0732.1","MA0733.1","MA0734.1","MA0735.1",
"MA0736.1","MA0737.1","MA0738.1",

"MA0739.1","MA0740.1","MA0741.1","MA0742.1","MA0743.1","MA0744.1","MA0745.1",
"MA0746.1","MA0747.1","MA0748.1",

"MA0749.1","MA0750.1","MA0751.1","MA0088.2","MA0752.1","MA0753.1","MA0754.1",
"MA0755.1","MA0756.1","MA0757.1",

"MA0469.2","MA0758.1","MA0473.2","MA0759.1","MA0760.1","MA0474.2","MA0098.3",
"MA0761.1","MA0762.1","MA0763.1",

"MA0764.1","MA0765.1","MA0156.2","MA0766.1","MA0767.1","MA0768.1","MA0769.1",
"MA0770.1","MA0771.1","MA0772.1",

"MA0052.3","MA0773.1","MA0498.2","MA0774.1","MA0775.1","MA0776.1","MA0777.1",
"MA0105.4","MA0778.1","MA0779.1",

"MA0780.1","MA0781.1","MA0782.1","MA0783.1","MA0784.1","MA0785.1","MA0786.1",
"MA0787.1","MA0788.1","MA0789.1",

"MA0790.1","MA0791.1","MA0792.1","MA0793.1","MA0794.1","MA0600.2","MA0795.1",
"MA0796.1","MA0797.1","MA0798.1",

"MA0799.1","MA0510.2","MA0511.2","MA0800.1","MA0801.1","MA0802.1","MA0803.1",
"MA0804.1","MA0805.1","MA0806.1",

"MA0807.1","MA0808.1","MA0809.1","MA0810.1","MA0003.3","MA0811.1","MA0812.1",
"MA0813.1","MA0524.2","MA0814.1",

"MA0815.1","MA0816.1","MA0461.2","MA0464.2","MA0817.1","MA0818.1","MA0819.1",
"MA0820.1","MA0821.1","MA0822.1",

"MA0823.1","MA0058.3","MA0825.1","MA0826.1","MA0827.1","MA0828.1","MA0829.1",
"MA0522.2","MA0830.1","MA0831.1",

"MA0832.1","MA0833.1","MA0834.1","MA0466.2","MA0836.1","MA0837.1","MA0838.1",
"MA0839.1","MA0840.1","MA0117.2",

"MA0841.1","MA0842.1","MA0843.1","MA0844.1","MA0845.1","MA0032.2","MA0846.1",
"MA0847.1","MA0848.1","MA0849.1",

"MA0850.1","MA0851.1","MA0852.1","MA0853.1","MA0854.1","MA0855.1","MA0856.1",
"MA0857.1","MA0858.1","MA0859.1",

"MA0860.1","MA0525.2","MA0106.3","MA0861.1","MA0862.1","MA0863.1","MA0024.3",
"MA0864.1","MA0865.1","MA0866.1",

"MA0867.1","MA0868.1","MA0869.1","MA0870.1","MA0871.1","MA0872.1","MA0028.2",
"MA0873.1","MA0874.1","MA0875.1",

"MA0876.1","MA0877.1","MA0878.1","MA0879.1","MA0880.1","MA0881.1","MA0882.1",
"MA0883.1","MA0884.1","MA0885.1",

"MA0886.1","MA0887.1","MA0888.1","MA0889.1","MA0890.1","MA0891.1","MA0892.1",
"MA0893.1","MA0894.1","MA0895.1",

"MA0896.1","MA0897.1","MA0898.1","MA0899.1","MA0900.1","MA0901.1","MA0902.1",
"MA0903.1","MA0904.1","MA0905.1",

"MA0906.1","MA0907.1","MA0908.1","MA0909.1","MA0910.1","MA0911.1","MA0912.1",
"MA0913.1","MA0914.1","MA1100.1",

"MA1101.1","MA0018.3","MA1102.1","MA0852.2","MA1103.1","MA0481.2","MA0036.3",
"MA1104.1","MA1105.1","MA1106.1",

"MA0039.3","MA1107.1","MA0495.2","MA0496.2","MA0620.2","MA1108.1","MA0147.3",
"MA0100.3","MA0104.4","MA1109.1",

"MA0161.2","MA0060.3","MA1110.1","MA1111.1","MA1112.1","MA0014.3","MA1113.1",
"MA1114.1","MA1115.1","MA0508.2",

"MA1116.1","MA1117.1","MA1118.1","MA1119.1","MA0442.2","MA1120.1","MA1121.1",
"MA1122.1","MA1123.1","MA0526.2",

"MA0750.2","MA0103.3","MA1124.1","MA1125.1","MA1154.1","MA1155.1","MA0037.3",
"MA0099.3","MA1126.1","MA1127.1",

"MA1128.1","MA1129.1","MA1130.1","MA1131.1","MA1132.1","MA1133.1","MA1134.1",
"MA1135.1","MA1136.1","MA1137.1",

"MA1138.1","MA1139.1","MA1140.1","MA1141.1","MA1142.1","MA1143.1","MA1144.1",
"MA1145.1","MA1146.1","MA1147.1",

"MA1148.1","MA1149.1","MA1150.1","MA1151.1","MA1152.1","MA0831.2","MA0693.2",
"MA1153.1","MA0162.3","MA1418.1",

"MA1419.1","MA1420.1","MA1421.1","MA1463.1","MA1464.1","MA1466.1","MA1467.1",
"MA1468.1","MA1101.2","MA1470.1",

"MA1471.1","MA1472.1","MA1473.1","MA1474.1","MA1475.1","MA1476.1","MA1478.1",
"MA1479.1","MA1480.1","MA1481.1",

"MA1483.1","MA1484.1","MA1485.1","MA1487.1","MA1489.1","MA1491.1","MA1493.1",
"MA1494.1","MA1495.1","MA1496.1",

"MA1497.1","MA1498.1","MA1499.1","MA1500.1","MA1501.1","MA1502.1","MA1503.1",
"MA1504.1","MA1505.1","MA1506.1",

"MA1507.1","MA1508.1","MA1509.1","MA1511.1","MA1512.1","MA1513.1","MA1514.1",
"MA1515.1","MA1516.1","MA1517.1",

"MA1518.1","MA1519.1","MA1520.1","MA1521.1","MA1522.1","MA1523.1","MA1524.1",
"MA1525.1","MA1527.1","MA1528.1",

"MA1529.1","MA1530.1","MA1531.1","MA1532.1","MA1533.1","MA1534.1","MA1535.1",
"MA1536.1","MA1537.1","MA1538.1",

"MA1539.1","MA1540.1","MA1541.1","MA1542.1","MA1544.1","MA1545.1","MA1546.1",
"MA1547.1","MA1548.1","MA1549.1",

"MA1550.1","MA1552.1","MA1553.1","MA1554.1","MA1555.1","MA1556.1","MA1557.1",
"MA1558.1","MA1559.1","MA1560.1",

"MA1561.1","MA1562.1","MA1563.1","MA1564.1","MA1565.1","MA1566.1","MA1567.1",
"MA1568.1","MA1569.1","MA1570.1",

"MA1571.1","MA1572.1","MA1573.1","MA1574.1","MA1575.1","MA1576.1","MA1577.1",
"MA1578.1","MA1579.1","MA1580.1",

"MA1581.1","MA1583.1","MA1584.1","MA1585.1","MA1587.1","MA1588.1","MA1589.1",
"MA1592.1","MA1593.1","MA1594.1",

"MA1596.1","MA1597.1","MA1599.1","MA1600.1","MA1601.1","MA1602.1","MA1603.1",
"MA1604.1","MA1606.1","MA1607.1",

"MA1608.1","MA1615.1","MA1616.1","MA1618.1","MA1619.1","MA1620.1","MA1623.1",
"MA1624.1","MA1625.1","MA1627.1",

"MA1628.1","MA1629.1","MA1630.1","MA1100.2","MA0605.2","MA0877.2","MA0462.2",
"MA0463.2","MA0878.2","MA0864.2",

"MA0469.3","MA0470.2","MA0612.2","MA0847.2","MA0766.2","MA0038.2","MA0734.2",
"MA0893.2","MA1099.2","MA0616.2",

"MA0650.2","MA0900.2","MA0158.2","MA0594.2","MA0902.2","MA0904.2","MA0485.2",
"MA0909.2","MA0912.2","MA0910.2",

"MA0913.2","MA1140.2","MA0701.2","MA0702.2","MA0703.2","MA0623.2","MA0842.2",
"MA0682.2","MA0075.3","MA0867.2",

"MA0868.2","MA0079.4","MA0516.2","MA0746.2","MA0632.2","MA0871.2","MA0753.2",
"MA0122.3","MA1707.1","MA1708.1",

"MA2093.1","MA2094.1","MA2095.1","MA1709.1","MA1710.1","MA2457.1","MA1711.1",
"MA1712.1","MA2096.1","MA1713.1",

"MA1714.1","MA1715.1","MA2097.1","MA1716.1","MA2098.1","MA2099.1","MA1717.1",
"MA1718.1","MA1719.1","MA1720.1",

"MA1721.1","MA2100.1","MA1722.1","MA1723.1","MA1724.1","MA1631.1","MA1632.1",
"MA1633.1","MA1634.1","MA1635.1",

"MA1636.1","MA1637.1","MA1638.1","MA1639.1","MA1640.1","MA1641.1","MA1642.1",
"MA1643.1","MA1644.1","MA1645.1",

"MA1646.1","MA1647.1","MA1648.1","MA1649.1","MA1650.1","MA1651.1","MA1652.1",
"MA1653.1","MA1654.1","MA1655.1",

"MA1656.1","MA1657.1","MA1683.1","MA2458.1","MA1725.1","MA1726.1","MA1727.1",
"MA1728.1","MA2101.1","MA1729.1",

"MA1730.1","MA1731.1","MA0833.2","MA0835.2","MA0465.2","MA0102.4","MA0836.2",
"MA0018.4","MA1102.2","MA0471.2",

"MA0154.4","MA0162.4","MA0598.3","MA0473.3","MA0640.2","MA0592.3","MA0761.2",
"MA0764.2","MA0765.2","MA0477.2",

"MA0148.4","MA0047.3","MA1103.2","MA0481.3","MA0062.3","MA0035.4","MA0482.2",
"MA1104.2","MA1105.2","MA0043.3",

"MA0114.4","MA0484.2","MA0901.2","MA0490.2","MA0491.2","MA0039.4","MA1107.2",
"MA0700.2","MA0495.3","MA0659.2",

"MA0496.3","MA0052.4","MA0620.3","MA1108.2","MA0499.2","MA0500.2","MA0056.2",
"MA0089.2","MA0025.2","MA0502.2",

"MA0063.2","MA1112.2","MA0679.2","MA0712.2","MA1113.2","MA0681.2","MA0782.2",
"MA0627.2","MA0508.3","MA0509.2",

"MA0798.2","MA0684.2","MA0743.2","MA0744.2","MA0745.2","MA0143.4","MA0080.5",
"MA0081.2","MA0829.2","MA0522.3",

"MA0830.2","MA0769.2","MA0090.3","MA0809.2","MA0003.4","MA0814.2","MA1123.2",
"MA0093.3","MA0526.3","MA0748.2",

"MA0528.2","MA0609.2","MA1684.1","MA2102.1","MA1928.1","MA1929.1","MA1930.1",
"MA1931.1","MA1932.1","MA1933.1",

"MA1934.1","MA1935.1","MA1936.1","MA1937.1","MA1938.1","MA1939.1","MA1940.1",
"MA1941.1","MA1942.1","MA1943.1",

"MA1944.1","MA1945.1","MA1946.1","MA1947.1","MA1948.1","MA1949.1","MA1950.1",
"MA1951.1","MA1952.1","MA1953.1",

"MA1954.1","MA1955.1","MA1956.1","MA1957.1","MA1958.1","MA1959.1","MA1960.1",
"MA1961.1","MA1962.1","MA1963.1",

"MA1964.1","MA1965.1","MA1966.1","MA1967.1","MA1968.1","MA1969.1","MA1970.1",
"MA1971.1","MA1972.1","MA1973.1",

"MA1974.1","MA1975.1","MA1976.1","MA1977.1","MA1978.1","MA1979.1","MA1980.1",
"MA1981.1","MA1982.1","MA1983.1",

"MA1984.1","MA1985.1","MA1986.1","MA1987.1","MA2117.1","MA2118.1","MA2119.1",
"MA2120.1","MA2121.1","MA2462.1",

"MA1988.1","MA1989.1","MA1990.1","MA1991.1","MA1992.1","MA1993.1","MA1994.1",
"MA1995.1","MA1996.1","MA1997.1",

"MA1998.1","MA1999.1","MA2001.1","MA2002.1","MA2122.1","MA2123.1","MA2124.1",
"MA2125.1","MA2126.1","MA2003.1",

"MA0037.4","MA0041.2","MA0050.3","MA0067.2","MA0078.2","MA0079.5","MA0080.6",
"MA0087.2","MA0095.3","MA0136.3",

"MA0152.2","MA0156.3","MA0157.3","MA0160.2","MA0466.3","MA0467.2","MA0474.3",
"MA0480.2","MA0489.2","MA0493.2",

"MA0505.2","MA0506.2","MA0507.2","MA0509.3","MA0514.2","MA0516.3","MA0521.2",
"MA0526.4","MA0606.2","MA0607.2",

"MA0611.2","MA0624.2","MA0625.2","MA0633.2","MA0642.2","MA0644.2","MA0650.3",
"MA0651.2","MA0659.3","MA0666.2",

"MA0668.2","MA0680.2","MA0685.2","MA0690.2","MA0693.3","MA0697.2","MA0707.2",
"MA0708.2","MA0723.2","MA0734.3",

"MA0740.2","MA0742.2","MA0754.2","MA0756.2","MA0759.2","MA0764.3","MA0765.3",
"MA0768.2","MA0784.2","MA0798.3",

"MA0799.2","MA0818.2","MA0819.2","MA0821.2","MA0828.2","MA0831.3","MA0837.2",
"MA0847.3","MA0865.2","MA0869.2",

"MA0877.3","MA0878.3","MA0879.2","MA0884.2","MA0885.2","MA0909.3","MA1110.2",
"MA1467.2","MA1472.2","MA1476.2",

"MA1483.2","MA1487.2","MA1491.2","MA1498.2","MA1511.2","MA1518.2","MA1524.2",
"MA1525.2","MA1532.2","MA1547.2",

"MA1563.2","MA1566.2","MA1567.2","MA1573.2","MA1601.2","MA1630.2","MA1633.2",
"MA1647.2","MA0597.2","MA1540.2",

"MA2323.1","MA2324.1","MA2325.1","MA2326.1","MA2469.1","MA2327.1","MA2328.1",
"MA2329.1","MA2330.1","MA2470.1",

"MA2331.1","MA2332.1","MA2333.1","MA2334.1","MA2335.1","MA2336.1","MA2337.1",
"MA2338.1","MA2339.1","MA2340.1",

"MA2341.1","MA0619.2","MA0006.2","MA0854.2","MA0634.2","MA0853.2","MA0690.3",
"MA0007.4","MA1463.2","MA0601.2",

"MA0602.2","MA0259.2","MA1464.2","MA0603.2","MA0874.2","MA1100.3","MA1631.2",
"MA1632.2","MA1988.2","MA0605.3",

"MA0833.3","MA1466.2","MA0834.2","MA0461.3","MA1467.3","MA0591.2","MA0496.4",
"MA1101.3","MA1470.2","MA0877.4",

"MA0635.2","MA0875.2","MA1471.2","MA1634.2","MA0462.3","MA0489.3","MA0488.2",
"MA0835.3","MA1989.2","MA0463.3",

"MA1472.3","MA1635.2","MA0817.2","MA0464.3","MA1928.2","MA0876.2","MA0465.3",
"MA1473.2","MA0102.5","MA0466.4",

"MA0836.3","MA0837.3","MA1636.2","MA0819.3","MA0018.5","MA0638.2","MA0839.2",
"MA1474.2","MA1475.2","MA0840.2",

"MA0609.3","MA0467.3","MA0139.2","MA1929.2","MA1930.2","MA1102.3","MA0754.3",
"MA0650.4","MA0901.3","MA0755.2",

"MA0639.2","MA0019.2","MA0879.3","MA0885.3","MA0880.2","MA0881.2","MA1476.3",
"MA0882.2","MA0883.2","MA1603.2",

"MA0610.2","MA1707.2","MA1478.2","MA2484.1","MA1479.2","MA1480.2","MA1481.2",
"MA0611.3","MA2485.1","MA0864.3",

"MA0469.4","MA0756.3","MA0470.3","MA0471.3","MA0865.3","MA0154.5","MA1604.2",
"MA1637.2","MA0162.5","MA0732.2",

"MA0733.2","MA0598.4","MA0473.4","MA1483.3","MA0640.3","MA0136.4","MA0028.3",
"MA0800.2","MA1708.2","MA1495.2",

"MA1932.2","MA0779.2","MA0014.4","MA0781.2","MA0686.2","MA1933.2","MA0828.3",
"MA0843.2","MA0759.3","MA0076.3",

"MA0612.3","MA0886.2","MA0027.3","MA0642.3","MA0760.2","MA1934.2","MA0820.2",
"MA1935.2","MA1936.2","MA0480.3",

"MA1937.2","MA0058.4","MA1938.2","MA0048.3","MA1939.2","MA0474.4","MA0112.4",
"MA0592.4","MA0141.4","MA0643.2",

"MA0644.3","MA0098.4","MA1484.2","MA0761.3","MA0762.2","MA1940.2","MA1941.2",
"MA1942.2","MA1943.2","MA0763.2",

"MA0764.4","MA0765.4","MA1944.2","MA1945.2","MA1946.2","MA1947.2","MA1948.2",
"MA0900.3","MA0645.2","MA0887.2",

"MA0888.2","MA0156.4","MA0475.3","MA1949.2","MA1950.2","MA0476.2","MA1951.2",
"MA0099.4","MA1126.2","MA1134.2",

"MA1140.3","MA0490.3","MA1141.2","MA0492.2","MA0491.3","MA2486.1","MA1135.2",
"MA0477.3","MA1128.2","MA1137.2",

"MA1142.2","MA1143.2","MA0478.2","MA1130.2","MA1131.2","MA1138.2","MA1139.2",
"MA1144.2","MA1145.2","MA0148.5",

"MA0047.4","MA1683.2","MA0846.2","MA0031.2","MA0847.4","MA0041.3","MA1487.3",
"MA1606.2","MA0030.2","MA0479.2",

"MA1952.2","MA0682.3","MA0851.2","MA0852.3","MA1103.3","MA1607.2","MA1953.2",
"MA1954.2","MA1955.2","MA1956.2",

"MA0157.4","MA0481.4","MA0593.2","MA0040.2","MA0062.4","MA0035.5","MA0140.3",
"MA0036.4","MA0037.5","MA0482.3",

"MA0766.3","MA1104.3","MA0889.2","MA0890.2","MA0646.2","MA0143.5","MA0767.2",
"MA0038.3","MA0483.2","MA1990.2",

"MA0734.4","MA1491.3","MA0735.2","MA0615.2","MA0647.2","MA1105.3","MA0648.2",
"MA0891.2","MA0892.2","MA0893.3",

"MA0092.2","MA0522.4","MA1638.2","MA1099.3","MA0616.3","MA0894.2","MA0649.2",
"MA0739.2","MA0738.2","MA1106.2",

"MA0131.3","MA0043.4","MA0895.2","MA0896.2","MA0897.2","MA0898.2","MA1991.2",
"MA0046.3","MA1494.2","MA0114.5",

"MA0484.3","MA0899.2","MA0911.2","MA1496.2","MA1497.2","MA1498.3","MA0594.3",
"MA0902.3","MA1566.3","MA0903.2",

"MA1499.2","MA0904.3","MA1500.2","MA1501.2","MA1502.2","MA1503.2","MA0905.2",
"MA0651.3","MA0906.2","MA0907.2",

"MA1504.2","MA1505.2","MA0485.3","MA1506.2","MA0908.2","MA0873.2","MA1958.2",
"MA0909.4","MA1507.2","MA0910.3",

"MA0913.3","MA1508.2","MA1992.2","MA0050.4","MA0051.2","MA1418.2","MA1419.2",
"MA0772.2","MA0652.2","MA1608.2",

"MA0914.2","MA0654.2","MA0656.2","MA1132.2","MA1133.2","MA0493.3","MA1512.2",
"MA0657.2","MA1513.2","MA1514.2",

"MA1515.2","MA1516.2","MA0039.5","MA1517.2","MA1959.2","MA1107.3","MA0618.2",
"MA0699.2","MA0768.3","MA1518.3",

"MA0700.3","MA0135.2","MA0704.2","MA1519.2","MA0658.2","MA0705.2","MA0701.3",
"MA0702.3","MA0703.3","MA1520.2",

"MA0501.2","MA0841.2","MA1521.2","MA0117.3","MA0495.4","MA0659.4","MA0089.3",
"MA0059.2","MA0147.4","MA1522.2",

"MA0029.2","MA0052.5","MA0497.2","MA0498.3","MA1639.2","MA1640.2","MA0775.2",
"MA0661.2","MA0706.2","MA1960.2",

"MA0620.4","MA0621.2","MA0662.2","MA0622.2","MA0664.2","MA0825.2","MA0707.3",
"MA1523.2","MA1524.3","MA0666.3",

"MA0708.3","MA0709.2","MA1108.3","MA0100.4","MA0104.5","MA1641.2","MA0499.3",
"MA0500.3","MA0056.3","MA1109.2",

"MA1993.2","MA0668.3","MA1642.2","MA0606.3","MA0624.3","MA0152.3","MA0625.3",
"MA1525.3","MA0150.3","MA0670.2",

"MA1643.2","MA0161.3","MA1527.2","MA0025.3","MA0671.2","MA1528.2","MA0778.2",
"MA0060.4","MA0502.3","MA1644.2",

"MA1529.2","MA1994.2","MA1645.2","MA0672.2","MA2003.2","MA0063.3","MA0673.2",
"MA0124.3","MA0122.4","MA0674.2",

"MA0675.2","MA1530.2","MA0125.2","MA0710.2","MA0626.2","MA1995.2","MA1531.2",
"MA1996.2","MA0494.2","MA1110.3",

"MA1146.2","MA1533.2","MA1534.2","MA1535.2","MA0504.2","MA1536.2","MA0164.2",
"MA0017.3","MA1537.2","MA1111.2",

"MA0677.2","MA0113.4","MA0727.2","MA1112.3","MA0160.3","MA1147.2","MA1540.3",
"MA0505.3","MA1541.2","MA0506.3",

"MA0842.3","MA1997.2","MA0679.3","MA0757.2","MA1542.2","MA1646.2","MA0711.2",
"MA0712.3","MA1544.2","MA1545.2",

"MA1961.2","MA0067.3","MA1546.2","MA0680.3","MA0070.2","MA1113.3","MA1114.2",
"MA0132.3","MA0681.3","MA0714.2",

"MA0782.3","MA1615.2","MA1548.2","MA0784.3","MA0785.2","MA0507.3","MA0627.3",
"MA0786.2","MA0790.2","MA0683.2",

"MA0791.2","MA1115.2","MA0628.2","MA1549.2","MA0793.2","MA1148.2","MA1550.2",
"MA0066.2","MA0065.3","MA0508.4",

"MA1998.2","MA1616.2","MA1647.3","MA1999.2","MA1723.2","MA0716.2","MA0075.4",
"MA1618.2","MA1619.2","MA1620.2",

"MA1149.2","MA1552.2","MA0859.2","MA1553.2","MA0718.2","MA0717.2","MA1116.2",
"MA1117.2","MA0138.3","MA0600.3",

"MA0799.3","MA0510.3","MA1724.2","MA1554.2","MA0629.2","MA0719.2","MA0072.2",
"MA1150.2","MA1151.2","MA0073.2",

"MA0002.3","MA0684.3","MA1963.2","MA0743.3","MA0744.3","MA0630.2","MA0720.2",
"MA1118.2","MA1119.2","MA0631.2",

"MA2001.2","MA1964.2","MA1153.2","MA1558.2","MA0745.3","MA1559.2","MA1560.2",
"MA0442.3","MA0869.3","MA1561.2",

"MA1120.2","MA1562.2","MA1152.2","MA0078.3","MA0514.3","MA0867.3","MA0087.3",
"MA0868.3","MA0077.2","MA0746.3",

"MA1965.2","MA0747.2","MA1564.2","MA0080.7","MA0081.3","MA0687.2","MA0829.3",
"MA0084.2","MA0137.4","MA0517.2",

"MA1623.2","MA0144.3","MA0518.2","MA1624.2","MA0519.2","MA1625.2","MA0520.2",
"MA0091.2","MA0108.3","MA0802.2",

"MA1565.2","MA0804.2","MA0688.2","MA1567.3","MA0521.3","MA1648.2","MA0832.2",
"MA1568.2","MA0830.3","MA0769.3",

"MA0523.2","MA0632.3","MA0090.4","MA1121.2","MA0809.3","MA0810.2","MA0003.5",
"MA0811.2","MA0812.2","MA0524.3",

"MA0814.3","MA1569.2","MA1966.2","MA1967.2","MA1968.2","MA1122.2","MA0692.2",
"MA0871.3","MA0597.3","MA1969.2",

"MA1574.2","MA1575.2","MA1576.2","MA1577.2","MA0861.2","MA1970.2","MA1123.3",
"MA0633.3","MA0721.2","MA0093.4",

"MA0526.5","MA0722.2","MA0723.3","MA0693.4","MA1578.2","MA0725.2","MA0726.2",
"MA1627.2","MA0844.2","MA0095.4",

"MA0748.3","MA0749.2","MA1971.2","MA1649.2","MA1650.2","MA0698.2","MA2487.1",
"MA1579.2","MA0527.2","MA2488.1",

"MA1581.2","MA0750.3","MA0694.2","MA0695.2","MA0103.4","MA2489.1","MA2490.1",
"MA2002.2","MA1651.2","MA1583.2",

"MA0146.3","MA1628.2","MA1629.2","MA0697.3","MA0751.2","MA1584.2","MA1709.2",
"MA1585.2","MA1973.2","MA1652.2",

"MA2491.1","MA1589.2","MA1653.2","MA1654.2","MA2492.1","MA1725.2","MA1974.2",
"MA1975.2","MA1710.2","MA0528.3",

"MA1592.2","MA1630.3","MA1154.2","MA1593.2","MA1976.2","MA2493.1","MA1977.2",
"MA1726.2","MA1655.2","MA1711.2",

"MA1978.2","MA1125.2","MA0752.2","MA1979.2","MA1727.2","MA2494.1","MA1656.2",
"MA1712.2","MA2495.1","MA1981.2",

"MA1728.2","MA1982.2","MA1983.2","MA2496.1","MA1713.2","MA2497.1","MA1657.2",
"MA1984.2","MA1714.2","MA1729.2",

"MA1599.2","MA1600.2","MA1986.2","MA1987.2","MA1730.2","MA0753.3","MA1716.2",
"MA1731.2","MA1717.2","MA1719.2",

"MA1720.2","MA1721.2","MA1602.2","MA1722.2","MA2503.1","MA2504.1","MA2506.1",
"MA2507.1","MA2508.1","MA2509.1",

"MA2510.1","MA2511.1","MA2512.1","MA2513.1","MA2514.1","MA2515.1","MA2516.1",
"MA2517.1","MA2518.1","MA2519.1",

"MA2520.1","MA2521.1","MA2522.1","MA2523.1","MA2524.1","MA2525.1","MA2526.1",
"MA2527.1","MA2528.1","MA2529.1",

"MA2530.1","MA2531.1","MA2532.1","MA2533.1","MA2534.1","MA2535.1","MA2536.1",
"MA2537.1","MA2538.1","MA2539.1",

"MA2540.1","MA2541.1","MA2542.1","MA2543.1","MA2544.1","MA2545.1","MA2546.1",
"MA2547.1","MA2548.1","MA2549.1",

"MA2550.1","MA2551.1","MA2552.1","MA2553.1","MA2554.1","MA2555.1","MA2556.1",
"MA2557.1","MA2558.1","MA2559.1",

"MA2560.1","MA2561.1","MA2562.1","MA2563.1","MA2564.1","MA2565.1","MA2566.1",
"MA2567.1","MA2568.1","MA2569.1",

"MA2570.1","MA2571.1","MA2572.1","MA2573.1","MA2574.1","MA2575.1","MA2576.1",
"MA2577.1","MA2578.1","MA2579.1",

"MA2580.1","MA2581.1","MA2582.1","MA2583.1","MA2584.1","MA2585.1","MA2586.1",
"MA2587.1","MA2588.1","MA2589.1",

"MA2590.1","MA2591.1","MA2592.1","MA2593.1","MA2594.1","MA2595.1","MA2596.1",
"MA2597.1","MA2598.1","MA2599.1",

"MA2600.1","MA2601.1","MA2602.1","MA2603.1","MA2628.1","MA2629.1","MA2674.1",
"MA2675.1","MA2676.1","MA2677.1",

```
"MA2678.1","MA2679.1","MA2680.1","MA2681.1","MA2682.1","MA2683.1","MA2684.1",
"MA2685.1","MA2686.1","MA2687.1",

"MA2688.1","MA2689.1","MA2690.1","MA2691.1","MA2692.1","MA0750.4","MA0695.3",
"MA2330.2","MA1589.3","MA0088.3", "MA1976.3","MA0116.2"
])

def motif_id_from_header(header: str) -> str:
  return header.lstrip(">").split()[0]

with open(jaspar_file, "r", encoding="utf-8") as fh:
  lines = fh.readlines()

out = []
i = 0
while i < len(lines):
  line = lines[i]
  if line.startswith(">"):
    motif_id = motif_id_from_header(line)
    block = lines[i:i+5]
    if motif_id in allowed_motif_ids:
      out.extend(block)
    i += 5
  else:
    i += 1

if not out:
  raise SystemExit("No motifs matched the whitelist.")

with open(filtered_out, "w", encoding="utf-8") as fh:
  fh.writelines(out)

print(f"Wrote filtered motif file with {len(out)//5} motifs to: {filtered_out}")
PY
##############################################################################
##########
# Merging BAMs
##############################################################################
##########
samtools merge -@ 8 "$OUT/merged_bam/0h_merged.bam" \
 "$BAMDIR/0hR1.dedup.bam" "$BAMDIR/0hR2.dedup.bam"
"$BAMDIR/0hR3.dedup.bam"
samtools sort -@ 8 -o "$OUT/merged_bam/0h_merged.sorted.bam"
"$OUT/merged_bam/0h_merged.bam"
samtools index "$OUT/merged_bam/0h_merged.sorted.bam"
```

```
samtools merge -@ 8 "$OUT/merged_bam/3h_merged.bam" \
  "$BAMDIR/3hR1.dedup.bam" "$BAMDIR/3hR2.dedup.bam"
"$BAMDIR/3hR3.dedup.bam"
samtools sort -@ 8 -o "$OUT/merged_bam/3h_merged.sorted.bam"
"$OUT/merged_bam/3h_merged.bam"
samtools index "$OUT/merged_bam/3h_merged.sorted.bam"

samtools merge -@ 8 "$OUT/merged_bam/12h_merged.bam" \
  "$BAMDIR/12hR1.dedup.bam" "$BAMDIR/12hR2.dedup.bam"
"$BAMDIR/12hR3.dedup.bam"
samtools sort -@ 8 -o "$OUT/merged_bam/12h_merged.sorted.bam"
"$OUT/merged_bam/12h_merged.bam"
samtools index "$OUT/merged_bam/12h_merged.sorted.bam"

samtools merge -@ 8 "$OUT/merged_bam/24h_merged.bam" \
  "$BAMDIR/24hR1.dedup.bam" "$BAMDIR/24hR2.dedup.bam"
"$BAMDIR/24hR3.dedup.bam"
samtools sort -@ 8 -o "$OUT/merged_bam/24h_merged.sorted.bam"
"$OUT/merged_bam/24h_merged.bam"
samtools index "$OUT/merged_bam/24h_merged.sorted.bam"

echo "All BAMs merged, sorted, and indexed"

##########################################################################
##########
# 3) TOBIAS ATACorrect
##########################################################################
##########
conda activate Tobias_env
cd "$TOBIAS_DIR"

TOBIAS ATACorrect \
  --bam "$OUT/merged_bam/0h_merged.sorted.bam" \
  --genome "$GENOME" \
  --peaks "$PEAKS" \
  --outdir "$OUT/ATACorrect/0h" \
  > "$OUT/logs/0h_ATACorrect.log" 2>&1

TOBIAS ATACorrect \
  --bam "$OUT/merged_bam/3h_merged.sorted.bam" \
  --genome "$GENOME" \
  --peaks "$PEAKS" \
  --outdir "$OUT/ATACorrect/3h" \
  > "$OUT/logs/3h_ATACorrect.log" 2>&1

TOBIAS ATACorrect \
  --bam "$OUT/merged_bam/12h_merged.sorted.bam" \
```

```
  --genome "$GENOME" \
  --peaks "$PEAKS" \
  --outdir "$OUT/ATACorrect/12h" \
  > "$OUT/logs/12h_ATACorrect.log" 2>&1

TOBIAS ATACorrect \
  --bam "$OUT/merged_bam/24h_merged.sorted.bam" \
  --genome "$GENOME" \
  --peaks "$PEAKS" \
  --outdir "$OUT/ATACorrect/24h" \
  > "$OUT/logs/24h_ATACorrect.log" 2>&1

echo "ATACorrect finished"

##########################################################################
##########
# 4) TOBIAS ScoreBigwig
##########################################################################
##########
TOBIAS ScoreBigwig \
  --signal "$OUT/ATACorrect/0h/0h_merged.sorted_corrected.bw" \
  --regions "$PEAKS" \
  --output "$OUT/ScoreBigwig/0h_footprints.bw"

TOBIAS ScoreBigwig \
  --signal "$OUT/ATACorrect/3h/3h_merged.sorted_corrected.bw" \
  --regions "$PEAKS" \
  --output "$OUT/ScoreBigwig/3h_footprints.bw"

TOBIAS ScoreBigwig \
  --signal "$OUT/ATACorrect/12h/12h_merged.sorted_corrected.bw" \
  --regions "$PEAKS" \
  --output "$OUT/ScoreBigwig/12h_footprints.bw"

TOBIAS ScoreBigwig \
  --signal "$OUT/ATACorrect/24h/24h_merged.sorted_corrected.bw" \
  --regions "$PEAKS" \
  --output "$OUT/ScoreBigwig/24h_footprints.bw"

echo "ScoreBigwig finished"

##########################################################################
##########
# 5) TOBIAS BINDetect using only the metazoan motifs
##########################################################################
##########
conda activate Tobias_env_bindetect
```

```
ulimit -n 8192

TOBIAS BINDetect \
 --cores 1 \
 --motifs "$FILTERED_MOTIFS" \
 --signals \
  "$OUT/ScoreBigwig/0h_footprints.bw" \
  "$OUT/ScoreBigwig/3h_footprints.bw" \
  "$OUT/ScoreBigwig/12h_footprints.bw" \
  "$OUT/ScoreBigwig/24h_footprints.bw" \
 --genome "$GENOME" \
 --peaks "$PEAKS" \
 --cond_names 0h 3h 12h 24h \
 --outdir "$OUT/BINDetect" \
 > "$OUT/BINDetect/bindetect.log" 2>&1

echo "BINDetect finished"

chmod +x run_tobias_whitelist.sh

bash -n run_tobias_whitelist.sh

./run_tobias_whitelist.sh

ls -lh /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/hypoxia_ATAC_peaks.animal_only_whitelist.jaspar

tail -n 50 /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/logs/0h_ATACorrect.log

ls -lh /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/ATACorrect/0h
ls -lh /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/ScoreBigwig
ls -lh /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/BINDetect

tail -n 50 /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia/BINDetect/bindetect.log
```

**Constructing genome track figures**

```
cd /drives/raid/AboobakerLab/James/hypoxia_ATAC-seq_time-course/april_2026_analysis/TOBIAS/TOBIAS_hypoxia

mkdir -p ATAC_bigwigs
```

```
conda install -c bioconda deeptools

bamCoverage -b merged_bam/0h_merged.sorted.bam -o ATAC_bigwigs/0h_ATAC.bw --normalizeUsing CPM --binSize 10 -p 8 --verbose

bamCoverage -b merged_bam/3h_merged.sorted.bam -o ATAC_bigwigs/3h_ATAC.bw --normalizeUsing CPM --binSize 10 -p 8 --verbose

bamCoverage -b merged_bam/12h_merged.sorted.bam -o ATAC_bigwigs/12h_ATAC.bw --normalizeUsing CPM --binSize 10 -p 8 --verbose

bamCoverage -b merged_bam/24h_merged.sorted.bam -o ATAC_bigwigs/24h_ATAC.bw --normalizeUsing CPM --binSize 10 -p 8 --verbose

grep "^#" ../../Smed_Rink.gtf > ../../Smed_Rink.sorted.gtf
grep -v "^#" ../../Smed_Rink.gtf | sort -k1,1 -k4,4n >> ../../Smed_Rink.sorted.gtf
```